Ders 19 Genişletilmiş Notu: Fonksiyon Çağrısı ve Stack, En Yakın Çift Problemi, Çöpçatan ve Doktor Senaryoları, Özyinelemeye Giriş

Ders 19 Genişletilmiş Notu: Fonksiyon Çağrısı ve Stack, En Yakın Çift Problemi, Çöpçatan ve Doktor Senaryoları, Özyinelemeye Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Önceki Dersten Hatırlatma: Fonksiyonlar Neden Yazılır?
- 2. Bir Fonksiyon Çağrısının Adımları
- 3. Fonksiyondan Dönünce Nereden Devam Edileceği Nasıl Bilinir?
- 4. Fonksiyon Kullanmanın Dezavantajları
- 5. Akış Diyagramında Fonksiyon, Overloading ve Farklı Adlar
- 6. En Yakın Çift (Closest Pair) Problemi
- 7. İki Boyutta Uzaklık
- 8. k Boyuta Genelleştirme
- 9. Çöpçatan Senaryosu
- 10. Kategorik Cevaplar ve findDistance Fonksiyonu
- 11. Doktor ve Teşhis Senaryosu
- 12. Özyinelemeli (Recursive) Fonksiyonlar
- 13. Özyinelemeli Faktöriyel
- 14. Üs Alma: Problemi Küçültmenin Kazancı

1. Önceki Dersten Hatırlatma: Fonksiyonlar Neden Yazılır?

Hoca derse önceki derste çözülen iki örneği hatırlatarak başladı. Birincisi, bir dizinin en büyük ya da en küçük elemanını bulan fonksiyondu. Hangisinin aranacağı fonksiyona bir seçim (flag) değişkeniyle bildiriliyordu. Fonksiyondan iki şey isteniyordu: elemanın değeri ve yeri. Değer `return` ile gönderiliyordu. C'de iki değer döndürülemediği için indis de pointer kullanılarak, call by reference benzeri bir mekanizmayla gönderiliyordu.

Hoca burada bir ayrım yaptı: C'de gerçek anlamda call by reference yoktur, Java'da da yoktur, C++ ve C# gibi dillerde vardır. C'deki mekanizma pointer yoluyla kurulan, "yalancı call by reference" denebilecek bir yapıdır. Birden fazla değer döndürmemizi sağlayan da budur.

İkinci örnekte matrisin en büyük ve en küçük elemanı bulunmuştu. Bunun için matrisin her satırı aynı fonksiyona gönderilmiş, satırların en büyük (ya da en küçük) elemanları bir dizide saklanmış, sonra bu dizi üzerinde yine aynı işlem yapılmıştı. Yani fonksiyon içinden fonksiyon çağrılmıştı.

Bu örneklerin amacı fonksiyonların avantajını göstermekti. Herkes en küçük-en büyük bulma işini `main` içinde de yazabilir. Ama fonksiyon bir kez yazılır ve istenildiği kadar çağrılır. Programlama dillerinin kendi içlerinde hazır fonksiyonlar bulundurmasının sebebi de budur: Kodu yazan kişi bunları tekrar tekrar sıfırdan yazmasın, etkili bir sürüm bir kez yazılsın ve tekrar

tekrar kullanılsın. Hoca şimdi bu mekanizmanın arka planına ve her şeyde olduğu gibi küçük dezavantajlarına bakacağını söyledi.

2. Bir Fonksiyon Çağrısının Adımları

Hoca tahtaya iki fonksiyon yazdı. İlki `int` döndüren ve `int n` alan bir faktöriyel fonksiyonu, ikincisi `int` bir dizi alan `void` bir `yaz` fonksiyonuydu.

Bir öğrenciden `main` içinde faktöriyel için örnek bir çağrı yazması istendi. Öğrenci önce yalnızca `fakt(5)` yazdı, sonra fonksiyon değer döndürdüğü için sonucun bir değişkene atanması gerektiğini söyledi: `a = fakt(5);`. `void` olan `yaz` için ise atama yapılmaz, doğrudan `yaz(a);` diye çağrılır. Hoca, normal bir dizi gönderirken eleman sayısının da gönderilmesi gerektiğini, ama burada bunun konuyu değiştirmediğini ekledi.

`a = fakt(5);` satırı çalışırken sırayla şunlar olur:

1. Parantez içindeki işlemler yapılır. Örneğin `fakt(5 + 5)` yazılsaydı önce toplama yapılırdı. Parantez içinde karmaşık işlemler de olabilir.
2. Parametreler fonksiyonun değişkenlerine kopyalanır. Birden fazla parametre varsa bu adımın alt adımları vardır. Örneğin `f(3, 4, 5)` çağrısında önce 3 `a`'ya (adım 2.1), sonra 4 (adım 2.2), sonra 5 aktarılır.
3. `return`'e kadar olan işlemler yapılır. Buradaki her algoritmik işlem bir alt adımdır.
4. Değer `return` edilir. Dönen değerin tipi fonksiyonun tipiyle uyumludur ve bu değer çağrının yerine düşer. `fakt(5)`'in sonucunun gelmesi, fonksiyonun tamamlandığını gösterir.
5. Atama yapılır: Değer `a` değişkenine kopyalanır.

   Hocanın vurgusu: Dördüncü adım önemlidir. `return` edilen şey, fonksiyonun çağrıldığı yere düşer.

`void` fonksiyonda ilk üç adım aynıdır. Fonksiyonun kapanışı görülünce dönülür. Bu dönüş dördüncü adım gibi düşünülebilir, ama hiçbir değer dönmez ve atama da yoktur. Program aşağıdan devam eder.

3. Fonksiyondan Dönünce Nereden Devam Edileceği Nasıl Bilinir?

3.1 Sorunun Ortaya Konması

Bu adımların arasında başka şeyler de olur. Hoca bunu göstermek için şu yapıyı kurdu: `main` içinde bazı satırlar var ve bir yerde `f` çağrılıyor. `f`'in içinde `g`, `g`'nin içinde `h` çağrılıyor. Satırlar numaralandırıldı: `main`'in 6. satırı `f`'i çağırıyor, program ardından 7. satırdan devam edecek.

Soru şuydu: `f` tamamlanıp geri gelindiğinde programın kaldığı yerden devam etmesi için, dili biz tasarlıyor olsaydık ne yapmamız gerekirdi?

Soru: `return` zaten bu işi yapmıyor mu? **Cevap:** `return` fonksiyondan dönmeyi sağlar, ama `void` bir fonksiyonda `return` hiç olmayabilir. Burada sorulan, döndükten sonra programın **hangi satırdan** devam edeceğinin nereden bilindiğidir.

Cevap şudur: Fonksiyon çağrısının olduğu satırı ya da ondan sonraki satırı (ikisi de olabilir) bir yerde saklamak gerekir. `f`'ten dönmek için 6. satır, `g`'den dönmek için `f`'in içindeki `g` satırı, `h`'ten dönmek için `g`'nin içindeki `h` satırı saklanmalıdır. `h`'in içinde en basitinden bir `printf` çağrılsa, `printf`'ten sonraki satır da saklanmalıdır.

Soru: Program zaten devam etsin istiyoruz. Bunu neden saklamamız gerekiyor? **Cevap:** Programı yazan kişi olarak biz saklamayacağız. Soruya derleyiciyi ya da programlama dilini tasarlayan kişi gözüyle bakıyoruz.

Soru: Her fonksiyonun kendi satırlarını saklayan bir değişkeni mi olacak? **Cevap:** Tam öyle değil, aşağıda görülecek.

Soru: Bir dizide saklayabilir miyiz? **Cevap:** Saklayabiliriz, ama boyutu sorun olur. Bu işlem sürekli tekrarlanıyorsa sonsuz bir dizi gerekir gibi görünür. Sonsuzluğu ise yakalayamayacağız.

Soru: Burada recursive bir şey mi yapıyoruz? **Cevap:** Hayır, ortada recursive bir şey yok. `main`'den `f`'e, `f`'ten `g`'ye, `g`'den `h`'e gidiyoruz. `h` bitince `g`, `g` bitince `f` bitiyor ve `main`'e dönülüyor.

Hoca bunu önceki derste yazılan kod üzerinden de izledi: `main`'den en küçük-en büyük bulma fonksiyonuna gidilir, bitince dönülür. Sonra `printf`'e gidilip dönülür, `scanf`'e gidilip dönülür. `for` ise gidilecek bir yer değildir, çünkü fonksiyon değildir. Matris için yazılan fonksiyona gidilince, onun içinden de dizi fonksiyonuna gidilir, döngü kaç kez çağırıyorsa o kadar gidilip dönülür.

Hoca bir benzetme yaptı: Yaşlılar eşyaları kaybolmasın diye onları hep bildikleri bir yerde, örneğin bir çekmecede saklar. Burada da programın sağlıklı devam etmesi için bazı önemli bilgileri belli bir yerde saklamamız gerekiyor.

3.2 Nerede Saklanır: Stack Bellek

Bu bilgiler RAM'de saklanır. Program çalıştırıldığında RAM'de ona bir yer ayrılır. Bu yerin içindeki bir alt bölgeye **stack memory** denir. Sınıftan gelen "stack" cevabı doğrudur, ama hoca kelimenin üç ayrı anlamda karşımıza çıktığını belirtti: veri yapısı olarak stack, işlemcinin içindeki stack pointer ve bellek bölgesi olarak stack memory.

> **Ders sonrası not:** Hoca stack pointer'ın 8 bitlik bir şey olduğunu söyledi. Doğrusu büyük olasılıkla şudur: Stack pointer, işlemcinin adres genişliğinde bir yazmaçtır (32 bitlik işlemcide 32, x86-64'te 64 bit). Bilginin kendisi yazmaçta değil RAM'dedir, stack pointer yalnızca o bölgenin tepesini gösterir. Hocanın asıl söylemek istediği, bu bilgilerin işlemcinin içine sığmayacağıdır, bu kısım değişmez.

3.3 Tek Değişken Neden Yetmez?

Programlar bellekte sıfır ve birler hâlinde, satır satır saklanır. Hoca adresleri şöyle varsaydı: `f` 100. satırda, `g` 200. satırda, `h` 300. satırda başlıyor. Her birinin sonunda bir `return` var. `main`'in 6. satırı `f`'i, `f`'in 105. satırı `g`'yi, `g`'nin 208. satırı `h`'i çağırıyor.

Dönüş satırını tek bir geçici değişkende tutmayı deneyelim. `f` çağrılınca 6 bu değişkene yazılır. Fonksiyon bitince 6 bir artırılır ve 7'den devam edilir. Tek fonksiyon için bu yeterlidir. Ama `f`'in içinde `g` çağrılınca aynı değişkene 105 yazmak gerekir. 105'e döneceğimiz garanti olur, fakat 6 kaybolur. Program `main`'e hiçbir şekilde dönemez ve bu fonksiyon bitince tamamen kapanır.

3.4 Çözüm: İlk Giren Son Çıkar

Çözüm, ilk gelenin son çıktığı **stack** veri yapısıdır. Hoca bunu dikey çizilmiş bir dizi gibi düşünmemizi istedi. Çağrılar geldikçe dönüş satırları sırayla üste eklenir, `return` görüldükçe en tepedeki çekilir:

Olay	Yapılan	Stack (alttan üste)	Devam edilen yer
6. satırda `f` çağrıldı	6 eklendi	6	100
105. satırda `g` çağrıldı	105 eklendi	6, 105	200
208. satırda `h` çağrıldı	208 eklendi	6, 105, 208	300
`h`'te `return`	208 çekildi	6, 105	208'den sonrası
`g`'de `return`	105 çekildi	6	105'ten sonrası
`f`'te `return`	6 çekildi	boş	6'dan sonrası

Devam etme işlemi işlemcideki **program counter** üzerinden olur. Hoca bunu C gibi yazarak gösterdi. `i` stack'in bir sonraki boş yerini göstersin (ara adım eklendi):

```
Fonksiyon çağrısı geldiğinde:  S[i++] = PC;
return görüldüğünde:           PC = S[--i];
```

Yani çağrıda program counter'ın içeriği stack'e kopyalanır, `return`'de stack'teki son eleman program counter'a yazılır ve indis bir azaltılır.

3.5 Yalnız Adres Saklanmaz

Stack'e yalnızca dönüş adresi yazılmaz. `main` içinde o ana kadar tanımlanmış diziler ve değişkenler vardır. Hoca bunların da stack'e kopyalandığını söyledi. Ayrıca işlemcinin o anki

yazmaçları, carry flag ve zero flag gibi durum (status) yazmaçları da stack'e kopyalanır. Fonksiyon bitince her şey yerine geri yüklenir: Adres program counter'a, durum yazmacı işlemciye, değişkenlerin değerleri de kendi yerlerine. Program akışı böylece sağlıklı devam eder. Bunların donanım tarafı Computer Organization dersinde daha ayrıntılı görülecek.

> **Ders sonrası not:** Hoca `main`'in değişkenlerinin çağrı sırasında stack'e kopyalanıp dönüşte geri yüklendiğini söyledi. Doğrusu büyük olasılıkla şudur: `main`'in yerel değişkenleri zaten stack'te, kendi çağrı alanında durur ve her çağrıda kopyalanmaz. Her çağrıda yeni bir alan açılır, oraya dönüş adresi, gerekli yazmaçlar, parametreler ve çağrılan fonksiyonun yerel değişkenleri yazılır.

Özetle, fonksiyon kullanan her programda böyle bir yedekleme mekanizması vardır ve bu mekanizma, çalışan programa (proses) tahsis edilen bellek bölgesinin içindeki stack memory alanını kullanır.

> **Hocanın vurgusu:** Fonksiyon çağrıları, prosese ayrılan belleğin içindeki stack bölgesinde yedeklenir. Bu önemli bir bilgidir.

4. Fonksiyon Kullanmanın Dezavantajları

Hoca bir öğrenciye bu mekanizmanın nasıl bir sorun çıkardığını sordu ve bir benzetmeyle yol gösterdi: Bir yolda gidiyorsun, yolda bir ara yere uğruyorsun. Fonksiyon yazmasaydık aynı kodla dümdüz devam edecektik, hiçbir yedekleme olmayacaktı. Öğrenci "fazladan zaman mı harcamış olduk?" dedi ve hoca doğruladı.

Birinci dezavantaj zaman kaybıdır. Yedekleyip geri almak zaman ister. Ama bu kayıp milisaniye ya da nanosaniye mertebesindedir ve ciddi bir kayıp oluşturmaz: İşlemci gigahertzlerle, bellek megahertzlerle çalışır. Önemli olan programın modüler hâle gelmesidir. Nesneye yönelik programlama dillerinde tasarım tamamen bu yapı üzerine kuruludur.

İkinci dezavantaj stack alanının sınırlı olmasıdır. İşletim sistemi bütün RAM'i bir prosese vermez, "senin yerin sınırlı" der. Bu sınırlı yerin içinde de yine sınırlı bir stack bölgesi vardır. Örneğin heap bölgesi belli bir sınıra kadar genişleyebilir, ama stack kesinlikle sınırlıdır. Çok fazla iç içe fonksiyon çağrılırsa **stack overflow** denen taşma hatası oluşur. Bu, dersin sonunda deneyle görüldü (Bölüm 12).

5. Akış Diyagramında Fonksiyon, Overloading ve Farklı Adlar

5.1 Akış Diyagramında Gösterim

Akış diyagramında bir fonksiyon **tanımlanırken** başlangıç sembolünün içine fonksiyonun adı ve giriş parametreleri, çıkışına da çıkış parametreleri yazılır. C'de `return` ile tek bir değer döner, ama pointer yardımıyla çok sayıda çıkış yapılabilir. Teorik olarak `a, b, c` diye istenildiği kadar çıkış yazılabilir.

Fonksiyon **çağrılırken** işlem kutusu, yani dikdörtgen kullanılır. Çünkü çağrının sonunda bir atama yapılabilir ya da hiçbir şey yapılmayabilir, ikisi de dikdörtgen içinde söylenebilir. Hoca bunu slayttaki en büyüğü bulma örneğiyle gösterdi: Alt programda fonksiyon adı ve girişleri, ana programda ise dikdörtgen içinde fonksiyon çağrısı vardı.

5.2 Fonksiyon Overloading (Aşırı Yükleme)

Overloading, aynı isimde birden fazla fonksiyon tanımlayabilmektir. Bu fonksiyonların aldığı parametreler farklı olmalıdır: parametrelerin sayısı, sırası ya da tipi. Hocanın örneği bir `yaz` fonksiyonuydu:

```
yaz(int a, char b[])
yaz(char b[], int a)
yaz(int a, int b, int c)
```

Bunlar farklı çalışan fonksiyonlardır. Derleyici, çağrıda gönderilene bakarak hangisinin çalışacağını otomatik olarak anlar: Örneğin ilk parametre olarak `"ABC"`, ikinci olarak 5 gönderilirse, karakter dizisi önce geldiği için ikinci tanım çağrılır. C'de bu mekanizma yoktur, Java gibi nesneye yönelik dillerde görülecektir.

Ders sonrası not: Hoca örneği önce `int` ve `char` ile kurdu, sonra `int` ile `char` birbirine dönüştürülebildiği için bunun olmayacağını söyleyip karakter dizisine geçti. Doğrusu büyük olasılıkla şudur: `f(int, char)` ile `f(char, int)` birlikte tanımlanabilir. Sorun yalnızca iki tanımın da aynı derecede uyduğu bir çağrıda belirsizlik çıkmasıdır. Karakter dizisi kullanmak bu belirsizliği ortadan kaldırır.

5.3 Aynı Kavramın Farklı Adları

Fonksiyonlara başka isimler de verilir. Hepsi aslında bir modüldür:

- **Function:** C'de kullanılan ad.
- **Method:** Daha çok Java ve C# gibi nesneye yönelik dillerde kullanılır.
- **Procedure:** Pascal'da kullanılan ad.
- **Subroutine (alt rutin):** Bir dile özgü değildir, daha kavramsal, genel bir addır.

6. En Yakın Çift (Closest Pair) Problemi

6.1 Öneri Sistemlerinin Arkasındaki Problem

Hoca konuya Spotify'ın haftalık keşif listesiyle girdi. Bu liste, dinlediğiniz müziklerden yola çıkarak benzer müzikler önerir. Peki bir öneri sistemi nasıl çalışır? Arkadaki teknik tarafın temelinde en yakın çift problemi vardır.

Hoca bunu doktora tezinden örneklerle gösterdi. Tezde müziklerin akustik özellikleri çıkarılıyor, bir müzik seçilince bu özelliklere göre en yakın 20 müzik öneriliyordu. Liste en yakından uzağa doğru sıralıdır. Bir müziğe en çok benzeyen, yüzde yüz benzerlikle kendisidir, onun ardından diğerleri gelir. Örnekler şunlardı:

- Bir klasik müzik seçildiğinde 20 önerinin 19'u başarılıydı, biri cazdı. Hoca cazın genelde klasiğe yakın bir müzik olduğunu söyledi.
- Rock seçildiğinde başarısız durumlar da görüldü: öneriler arasında bir reggae, bir country, birkaç metal vardı.
- Chopin'den bir eser seçildiğinde Beethoven ve yine Chopin önerildi.
- Türk müziği karışık listesinde en başarılı sonuçlardan biri mehter marşlarında alınmıştı. Bir şarkıcının bir şarkısı seçildiğinde Gülpembe ve Dönence gibi şarkılar önerildi.

Bu veri seti yaklaşık bin müziklikti. Daha gerçekçi bir deneme için Turkcell ile çalışıldı ve onların müzik veri seti kullanıldı. Bu veri setinde bir müzik seçildiğinde ilk sıralarda aynı şarkıcının müzikleri gelmişti.

Hoca mantığı bir atasözüyle özetledi: "Bana arkadaşını söyle, sana kim olduğunu söyleyeyim." En yakınındaki kimse, sen de ona çok benzersin.

6.2 Tek Boyutlu Örnek

Önce tek boyutta düşünelim. Sayı doğrusu üzerinde 0'dan 8'e kadar değerler var. Dizinin elemanları önce 3, 4, 8, 7, 5, 2 olarak yazıldı. Hoca farklı bir örnek çıksın diye 2'yi 1 yaptı. Dizi şöyle oldu: 3, 4, 8, 7, 5, 1.

Verilen: `A = {3, 4, 8, 7, 5, 1}`. **İstenen:** Birbirine en yakın iki eleman. Uzaklık, iki sayının farkının mutlak değeridir. Her eleman diğerleriyle karşılaştırılır. Hoca sayı doğrusunda küçükten büyüğe gitti:

- **1 için:** 3 ile uzaklık 2, 4 ile 3, 5 ile 4. En yakını 1-3 ikilisi (uzaklık 2).
- **3 için:** 1 ile uzaklık 2, 4 ile uzaklık 1. Daha yakın bir ikili bulundu: 3-4 (uzaklık 1).
- **4 için:** 1 ile 3, 3 ile 1, 5 ile 1. Yani 4-5 de uzaklığı 1 olan bir ikilidir.
- **5, 7 ve 8 için:** Aynı işlem yapılır. 7-8 ikilisinin uzaklığı da 1'dir.

Sonuç: En yakın ikililer 3-4, 4-5 ve 7-8'dir, uzaklıkları 1'dir.

6.3 Benzemek Ne Demek?

Hoca problemi günlük hayata bağlamak için iki öğrenciye ailede kime benzetildiklerini sordu. Biri görünüş olarak yüz şekliyle annesine, diğeri huy olarak çalışkanlık ve kararlılıkla dayısına

benzetildiğini söyledi.

Yapılan benzetme aslında şudur: "Yüz şekli" gibi bir özelliği bir sayıyla ifade edip bir sayı doğrusuna yazın. Doğrunun üzerinde anne, ben, baba ve kardeş birer nokta olur. Benimle annem arasındaki uzaklığa, benimle babam arasındaki uzaklığa bakılır. En küçük uzaklık kimdeyse en çok ona benzerim. Kararlılık için de ayrı bir doğru çizilir, üzerinde dayı, ben, anne, dede bulunur ve en yakın olan bulunur. Her özelliği bir sayı gibi düşündüğümüzde en yakını bulma problemi budur.

6.4 Tek Boyutlu Algoritma

Bir eleman için kalan elemanlara, sonra bir sonraki eleman için kalan elemanlara bakacağız. Bu, iç içe iki `for` döngüsü demektir ve `n` elemanlı bir dizide `n²` mertebesinde işlem gerektirir.

> **Ders sonrası not:** Hoca karşılaştırma sayısının $\frac{n(n+1)}{2}$ olduğunu söyledi. `j` değişkeni `i + 1`'den başladığında doğrusu $\frac{n(n-1)}{2}$'dir. Dersteki 6 elemanlı dizi için $\frac{6 \cdot 5}{2} = 15$ çift vardır (python ile sayıldı). Mertebe yine n^2'dir.

Hoca n^2'den kurtulan bir çözüm olduğunu, işin $n \log n$'e indirilebildiğini söyledi. Turkcell verisi gibi çok büyük veri setlerinde benzerlik hesabını n^2 kez yapmayan algoritmalar kullanılır. Bu derste n^2'lik çözüm yazılacak.

> **Ek bilgi (derste anlatılmadı):** Tek boyutta $n \log n$ sonucu büyük olasılıkla şöyle elde edilir: Dizi önce sıralanır, en yakın çift bu durumda mutlaka yan yana iki eleman olur ve tek bir taramayla bulunur. Çok boyutlu durumda ise farklı yöntemler gerekir.

Akış diyagramı `EnYakin(A, n)` diye bir alt programdır. Girişleri `A` dizisi ve eleman sayısı `n`, çıkışları `indis1` ve `indis2`'dir. Hoca en küçüğü bulurken başlangıç değerinin ilk iki elemanın farkı olarak da seçilebileceğini, ama buna gerek olmadığını söyledi: Başlangıçta `minDist` sonsuz kabul edilir. Adımlar şöyledir:

1. `minDist = sonsuz`, `indis1 = indis2 = 0` yapılır.
2. `i`, 0'dan `n-2`'ye birer birer gider. Son elemana bakılmaz, çünkü ondan sonra karşılaştırılacak eleman kalmaz.
3. `j`, `i+1`'den `n-1`'e birer birer gider.
4. `dist = |A[i] - A[j]|` hesaplanır.
5. `dist < minDist` ise `minDist = dist`, `indis1 = i`, `indis2 = j` yapılır.
6. `j` döngüsü, ardından `i` döngüsü tamamlanınca alt programdan `return` ile çıkılır.

`j`'nin nereden başlayacağını hoca bir öğrenciye sordu. Cevap `i + 1` oldu. Çünkü `j`, `i`'den başlatılsa eleman kendisiyle karşılaştırılırdı, buna gerek yoktur. Önceki çiftlere de zaten bakılmıştır.

Hoca bu tek boyutlu hâlin kodunu yazmadı, "bunu yazamayacak kimse yoktur" dedi. Tek dikkat edilecek nokta, `i1` ve `i2`'nin birlikte `return` edilememesidir: Biri pointer, biri `return` ile ya da ikisi de pointer ile döndürülür.

7. İki Boyutta Uzaklık

Her sayının yerine iki karakteristik özelliği olan noktalar düşünelim. Hocanın örneğinde `x` göz rengi, `y` saç rengi olsun ve renkler 0 ile 255 arasında sayısal olarak kodlansın. Bir nokta ben, biri dayım, biri annem, biri dedem olur. Veri, `n` satırlı ve 2 sütunlu bir `A` matrisinde tutulur: Satırlar noktaları (kişileri), sütunlar özellikleri gösterir. Algoritma aynıdır: Bir nokta referans alınır, diğerleriyle arasındaki uzaklığa bakılır. Değişen tek şey uzaklık hesabıdır.

Soru: İki boyutta uzaklık nasıl hesaplanır? **Cevap:** Bir öğrenci farkları toplamayı önerdi. Hoca bunu, `x`'ler arasındaki farkın mutlak değeriyle `y`'ler arasındaki farkın mutlak değerinin toplanması olarak netleştirdi. Bu **Manhattan uzaklığıdır** ve L1 normu olarak da adlandırılır:

$$d = |A[i][0] - A[j][0]| + |A[i][1] - A[j][1]|$$

En kısa uzaklığa bakmak istersek Pisagor bağıntısından gelen **Öklid uzaklığı** kullanılır. Farkların kareleri toplanır ve toplamın karekökü alınır:

$$d = \sqrt{(A[i][0] - A[j][0])^2 + (A[i][1] - A[j][1])^2}$$

Soru: Eğim ya da türev kullanılabilir mi? **Cevap:** Hayır. Eğim bir artış ya da azalışı gösterir, yakınlık veya uzaklık göstermez. Örneğin 1 ile 5 arasındaki eğim ve 1 ile 2 arasındaki eğim aynı olabilir, ama uzaklıklar farklıdır. Türev de değişimi gösterir ve uzaklık belirtmez.

Koda geçerken döngüler aynı kalır. `A` artık dizi değil, `n` satırlı ve 2 sütunlu bir matristir ve yalnızca uzaklık satırı değişir.

8. k Boyuta Genelleştirme

8.1 Veri Yapısı ve Algoritma

Gerçek hayatta her şey iki boyutta çözülmez. Noktanın `k` özelliği varsa matris `n` satır ve `k` sütundan oluşur. Satırlar örnekleri (0'dan `n-1`'e), sütunlar özellikleri (0'dan `k-1`'e) gösterir. Hoca bunun iki boyutta görselleştirilebileceğini, ama `k` boyutun çizilemeyeceğini söyledi. Çizemesek de bilgisayarda `n × k`'lık bir matrisle saklayabiliriz.

Uzaklık hesabında artık sıfırıncı sütunla sıfırıncı, birinciyle birinci, ikinciyle ikinci karşılaştırılır. Bunun için `k` kez dönen üçüncü bir döngü gerekir. `Mindist(A, n, k)` alt programının adımları şöyledir:

1. `minDist = sonsuz`, `i1 = i2 = 0`, `dist = 0` yapılır.
2. `i`, 0'dan `n-2`'ye birer birer gider.
3. `j`, `i+1`'den `n-1`'e birer birer gider.
4. `dist = 0` yapılır. Bu sıfırlama her iki örnek için tekrar gerekir.
5. `l`, 0'dan `k-1`'e birer birer gider: `dist += |A[i][l] - A[j][l]|`.
6. `l` döngüsü bitince `dist < minDist` ise `minDist = dist`, `i1 = i`, `i2 = j` yapılır.
7. `j` döngüsü, ardından `i` döngüsü biter ve `return` ile çıkılır.

Hoca çizerken beşinci adımda indis olarak `k` yazdığını fark edip `l` olarak düzeltti, çünkü döngü değişkeni `l`'dir. Karşılaştırma da `l` döngüsünün **içine** değil, bittikten sonrasına yerleştirilir: Önce `k` özelliğin farkları toplanır, sonra elde edilen uzaklıkla karşılaştırma yapılır.

Öklid uzaklığı için beşinci adımda `dist += (A[i][l] - A[j][l])²` yazılır ve döngü bitince `dist = √dist` yapılır.

Soru: Karekök almaya gerek var mı? Kareler toplamı en küçük olan, karekökü alındığında da en küçük olur. **Cevap:** Doğru. Yalnızca en yakını bulacaksak karekök almaya gerek yoktur. Tam Öklid uzaklığının değeri istenirse alınır.

Soru: Sol üstteki matriste satırlar neyi temsil ediyor? **Cevap:** Satırlar örnek sayısını, yani kişi ya da nokta sayısını söyler. Sütunlar da her noktanın kaç özelliği, yani boyutu olduğunu söyler.

8.2 İkinci, Üçüncü En Yakını Bulmak

Soru: Yalnız en yakını değil, ikinci ve üçüncü en yakını da bulmak istesek ne yaparız? **Cevap:** En yakın ikilileri bir dizi yerine bir matriste tutarız. Her satır sırayla birinci, ikinci, üçüncü en yakın ikiliyi tutar. Örneğin ilk satırda en yakın ikilinin indisleri, diyelim 3 ve 5, yazılır. Hocanın daha güzel bulduğu sürümde uzaklık değeri de ayrı bir sütuna yazılır. Yeni bir uzaklık hesaplanınca tablodaki uzaklıklarla karşılaştırılır. Daha küçükse büyükler aşağı kaydırılır ve yeni değer yerine yazılır. Böylece örneğin en yakın 10 ikili bir matriste tutulabilir. Hoca bunun kodlanırsa güzel bir uygulama olacağını söyledi. İki tane için iki değişken belki yeterlidir, ama genelleştirmek için matris gerekir.

8.3 Nerede Kullanılır?

Müzik örneği de budur: Bir milyon müzik ve `n` özellik varsa bir milyon çarpı `n`'lik bir matris tasarlanır. En basit örneklerden biri de çöpçatan siteleridir. `n` kişiye `m` soru sorulur (yaşın kaç, maaşın kaç gibi). Sorulara verilen sayısal cevapların benzerliğinden yola çıkılarak kimin kimle arkadaş olabileceği bulunur.

Burada önemli bir ayrıntı vardır: özelliklerin ölçeği. Yaş 20 ile 70, maaş 1000 ile 20000 arasında olsun. Fark hesabı yapılınca maaşlar arasındaki fark yaşlar arasındaki farkı bastırır ve yaşın bir önemi kalmamış gibi görünür. Bu durumda özellikler belli bir aralığa normalize edilir, yani ölçeklenir. Hoca bu kısmın istatistik ve veri madenciliği tarafı olduğunu, burada yalnızca algoritmanın yaptığı işe bakılacağını söyledi.

> **Ek bilgi (derste anlatılmadı):** Normalizasyonun yaygın bir biçimi büyük olasılıkla min-max ölçeklemedir: $x' = \frac{x - x_{min}}{x_{max} - x_{min}}$. Bu dönüşümle bütün özellikler 0 ile 1 arasına gelir ve birinin diğerini bastırması önlenir.

9. Çöpçatan Senaryosu

9.1 Problem ve Veri Yapısı

Hoca genel hâli bir senaryoyla kodladı. Elimizde `n` kişi var, ama erkekler ve kadınlar iki ayrı matriste tutuluyor. Erkek matrisi `erkek sayısı × soru sayısı`, kadın matrisi `kadın sayısı × soru sayısı` boyutundadır. Herkese aynı sorular sorulur, bu yüzden sütun sayısı aynıdır. Satır sayıları farklı olabilir. Önceki bölümden farkı, benzerliğe tek bir matrisin içinde değil, iki matris arasında bakılmasıdır. Yine iç içe üç döngü olacaktır.

> **Hocanın vurgusu:** Bu, sınavda karşınıza gelebilecek bir senaryodur. Yapılan iş matris işlemidir, ama soru doğrudan öyle sorulmaz. Uygun veri yapısını sizin seçmeniz gerekir.

Boyutlar `#define` ile parametrik yazıldı. Başlangıçta 10 erkek, 15 kadın ve 20 soruydu, sonra deneme için 4 erkek, 6 kadın, 5 soruya, elle kontrol edebilmek için de 2 erkek, 3 kadın, 3 soruya indirildi. Cevaplar 1 ile 5 arasında rastgele verilir.

9.2 Kod: Matrisin Doldurulması ve En Uygun İkili

Matrisi doldurmak için bir fonksiyon yazıldı. Matris parametresinde sütun sayısı yazılmak zorundadır, satır sayısı ise parametre olarak gelir. Bu sayede aynı fonksiyon hem erkek hem kadın matrisi için kullanılabilir.

Soru: Döngüde üst sınır olarak erkek sayısı yazarsak kadınlar için ne olacak? **Cevap:** Bu yüzden satır sayısı `n` parametresiyle alındı. `matris_doldur(E, ERKEK)` ve `matris_doldur(K, KADIN)` diye çağrılır.

Soru: Fonksiyonda hem `M` matrisi hem `M` tam sayısı var. Sorun çıkarır mı? **Cevap:** Evet, çıkarır. Değişken adlarının birbirinden farklı olması lazım. Hoca birini büyük harfle bırakıp diğerini küçük harf yaptı: `M` matris, `m` tam sayı.

Hoca, soru sayısı kullanıcıdan alınacaksa matrisin örneğin 50 sütunlu tanımlanıp yalnız gereken kısmının kullanılabileceğini de hatırlattı. Değişkenleri alt alta yazıp yanına açıklama eklemenin kodun okunabilirliğini artırdığını söyledi.

Rastgele sayı üreteci `srand(time(NULL))` ile başlatılır. Hoca doktor programına geçerken bu çağrıyı programın en başına aldı.

> **Ek bilgi (derste anlatılmadı):** `srand` programda büyük olasılıkla yalnızca bir kez çağrılmalıdır. `time(NULL)` saniye çözünürlüklü olduğundan, `srand` her doldurma fonksiyonunda çağrılırsa aynı saniyedeki çağrılar aynı rastgele diziyi üretir.

En uygun ikiliyi bulan `enUygunBul` fonksiyonu `void` tiptedir. İki indisi `i1` ve `i2` pointer'ları üzerinden döndürür, bu yüzden `main`'de `&i1, &i2` gönderilir. Tam kodun ilk kısmı şöyledir, kalanı Bölüm 10'dadır:

```c
#include <stdio.h>
#include <stdlib.h>   /* rand, srand, abs */
#include <time.h>     /* time */
#include <limits.h>   /* INT_MAX */

#define SORU 3
#define ERKEK 2
#define KADIN 3
```

```c
void matris_doldur(int M[][SORU], int n);
int findDistance(int x, int y, char c);
void enUygunBul(int E[][SORU], int n, int K[][SORU], int m,
                int *i1, int *i2);
void enUygunBul2(int E[][SORU], int n, int K[][SORU], int m,
                 char c, int *i1, int *i2);

void matris_doldur(int M[][SORU], int n)
{
    int i, j;
    for (i = 0; i < n; i++) {
        for (j = 0; j < SORU; j++) {
            M[i][j] = 1 + rand() % 5;   /* 1 ile 5 arasinda cevap */
            printf("%2d ", M[i][j]);
        }
        printf("\n");
    }
}

void enUygunBul(int E[][SORU], int n, int K[][SORU], int m,
                int *i1, int *i2)
{
    int i, j, k;             /* dongu degiskenleri */
    int min_d = INT_MAX;     /* simdiye kadarki en kucuk uzaklik */
    int dist;                /* o anki ikilinin uzakligi */

    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {      /* iki ayri matris */
            dist = 0;                  /* her ikili icin sifirlanir */
            for (k = 0; k < SORU; k++)
                dist += abs(E[i][k] - K[j][k]);
            if (dist < min_d) {
                min_d = dist;
                *i1 = i;
                *i2 = j;
            }
        }
    }
}
```

Hoca koddaki kararları tek tek açıkladı:

- **Sonsuz yerine `INT_MAX`:** Kodda sonsuz yazılamaz. `INT_MAX`, `int`'in bu bilgisayarda alabileceği en büyük değerdir. İki cevap arasındaki farklar buna hiç ulaşmayacağı için güvenle kullanılabilir.
- **`j` neden 0'dan başlar?** Önceki örnekte tek matris vardı ve `j`, `i + 1`'den başlıyordu. Burada iki farklı matris olduğu için her erkek her kadınla karşılaştırılmalıdır.
- **Döngülerin sırası:** Erkekle kadın arasındaki fark, kadınla erkek arasındaki farka eşittir. Bu simetrik olduğu için döngüler yer değiştirebilir.
- **`dist` nerede sıfırlanır?** Hoca önce sıfırlamayı karşılaştırmadan sonraya yazdı, sonra iç döngüye girmeden önce sıfırlamanın daha iyi olduğunu söyledi.
- **`dist += ...` yazımı:** Uzun yazılırsa `dist = dist + ...` demektir.

Bir öğrenci koddaki bir yazım hatasını yakaladı: İç döngüde kadın matrisi için `K[j][j]` yazılmıştı, doğrusu `K[j][k]` olarak düzeltildi. Sorular yalnız iki tane olsaydı üçüncü döngüye gerek kalmazdı.

> **Ders sonrası not:** Hoca `abs` için `math.h` eklenmesi gerektiğini söyledi. `abs` tam sayı fonksiyonudur ve `stdlib.h`'te bildirilir. Yalnız `math.h` eklenince `gcc -Wall` "implicit declaration" iletisi verir: C89 modunda uyarı, C99 modunda bu derleyicide hata olarak (denendi). `math.h`'teki karşılık `fabs`'tır ve büyük olasılıkla `double` ile çalışır.

`INT_MAX` da `limits.h`'tedir.

9.3 Sonucun Elle Kontrolü

2 erkek, 3 kadın ve 3 soruyla çalıştırılan programda ekrana en uygun kişiler olarak 1 ve 2 geldi. Hoca sonucu gözle kontrol etti. Her ikili için soruların farklarını topladı ve bir ikilinin toplamı 2'yi geçince kalanına bakmaya gerek olmadığını belirtti:

Erkek	Kadın	Okunan farklar	Toplam uzaklık
0	0	2, 1, 3	6
0	1	0, 2, ?	2
0	2	1, 4, …	7
1	0	2, 3, …	7
1	1	0, 2, …	3
1	2	1, 0, 1	2

En küçük uzaklık 2'dir, yani en yüksek benzerlik bu ikililerdedir.

> **Ders sonrası not:** Tablodaki okumaya göre uzaklığı 2 olan iki ikili var: (0, 1) ve (1, 2). `dist < min_d` karşılaştırması eşitlikte güncelleme yapmadığı için önce bulunan (0, 1) kalmalıdır. Aynı farkları veren sabit verilerle kod çalıştırıldığında gerçekten `0 1` yazdı. Oysa hoca ekranda 1 ve 2 okudu. Farkın sebebi altyazıdan anlaşılmıyor, büyük olasılıkla ya ikinci satırın okuması ya da ekrandaki kod farklıydı.

10. Kategorik Cevaplar ve findDistance Fonksiyonu

10.1 Kategorik Uzaklık

Cevaplar 1 ile 5 arasındaki şıklardan biri olsun, ama aralarındaki fark anlamlı olmasın. Bu durumda cevaplar **kategoriktir**. 1 ile 2 arasındaki farkın 1, 1 ile 3 arasındaki farkın 2 olmasının bir anlamı yoktur. Benzerlik hesabı değişmelidir.

Bir öğrenci aynı olan cevapların sayısına bakmayı önerdi ve hoca bunu örnekle gösterdi:

- **Verilen:** Birinci kişinin cevapları 1, 2, 3, 4, 5; ikinci kişinin cevapları 2, 2, 3, 4, 1.
- **İşlem:** 1 ve 2 eşit değil, 2 ve 2 eşit, 3 ve 3 eşit, 4 ve 4 eşit, 5 ve 1 eşit değil.
- **Sonuç:** Benzerlik 3 birimdir, kişiler iki sorudan uyuşmamaktadır, yani uzaklık 2'dir.

En yüksek benzerlik ya da en küçük uzaklık hangi ikilideyse o seçilir. Benzerlik ve uzaklık birbirinin tam tersidir. Uzaklık olarak yazarsak aynı cevap 0, farklı cevap 1 sayılır. Birinin 1-2, diğerinin 1-5 gibi farklı cevaplar vermesi önemli değildir, her ikisi de tek bir uyuşmazlıktır.

10.2 Kod

Hoca aynı fonksiyona bir `char c` parametresi eklenmiş ikinci bir sürüm yazdı ve uzaklık hesabını ayrı bir fonksiyona aldı: `c` ile kategorik, başka bir şeyle (`n` ile) sayısal çalışır. `findDistance`'ın parametreleri matris değil, matrisin iki elemanıdır, yani iki `int` ve bir `char`'dır. Bir öğrenci fonksiyona önce matris yazılacağını söyledi, hoca gönderilene dikkat etmesini istedi.

Soru: Burada neden pointer kullanmadık? **Cevap:** Çünkü gönderilen değerleri değiştirmek istemiyoruz. Yalnızca hesaplayıp `return` ile sonuç döndürüyoruz.

Kodun kalanı şöyledir:

```
int findDistance(int x, int y, char c)
{
    if (c == 'c') {          /* kategorik */
        if (x == y)
            return 0;
        else
            return 1;
    }
```

```
    return abs(x - y);       /* sayisal */
}

void enUygunBul2(int E[][SORU], int n, int K[][SORU], int m,
                 char c, int *i1, int *i2)
{
    int i, j, k;
    int min_d = INT_MAX;
    int dist;

    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            dist = 0;
            for (k = 0; k < SORU; k++)
                dist += findDistance(E[i][k], K[j][k], c);
            if (dist < min_d) {
                min_d = dist;
                *i1 = i;
                *i2 = j;
            }
        }
    }
}

int main(void)
{
    int E[ERKEK][SORU];   /* erkeklerin cevaplari */
    int K[KADIN][SORU];   /* kadinlarin cevaplari */
    int i1, i2;           /* en uygun ikilinin indisleri */

    srand(time(NULL));

    matris_doldur(E, ERKEK);
    printf("\n");
    matris_doldur(K, KADIN);
    printf("\n");

    enUygunBul(E, ERKEK, K, KADIN, &i1, &i2);
    printf("En uygun kisiler (sayisal): %d %d\n", i1, i2);

    enUygunBul2(E, ERKEK, K, KADIN, 'n', &i1, &i2);
    printf("En uygun kisiler (versiyon 2, sayisal): %d %d\n", i1, i2);

    enUygunBul2(E, ERKEK, K, KADIN, 'c', &i1, &i2);
    printf("En uygun kisiler (versiyon 2, kategorik): %d %d\n", i1, i2);

    return 0;
}
```

Bu iki kod parçası tek dosyada birleştirilip `gcc -Wall -std=c89 -pedantic` ile uyarısız derlendi.

Burada `return`'ün önemli bir kullanımı görülüyor: Fonksiyonun ortasında da `return` yazılabilir. `return` çalışınca fonksiyonun geri kalanı çalışmaz. Eşitlik doğruysa fonksiyon diğer işlemlere bakmadan 0 döndürür.

Hoca ikinci sürümün sayısal modda birinci sürümle aynı sonucu verdiğini de denedi ve sonuçlar aynı çıktı. Bu yüzden `enUygunBul`'a artık pek gerek kalmaz, çünkü ikinci sürüm daha geneldir. Yazım sırasında çıkan iki hata da öğreticiydi: `findDistance`'a farkın eksi işaretiyle değil, virgülle ayrılmış iki parametre olarak gönderilmesi gerekiyordu. Ayrıca `abs(x - y)` satırının başına `return` yazılmamıştı.

Ders sonrası not: Hoca `if` kolu için `return` yazıldıysa `else` kolu için de yazılması ge-

rektiğini, aksi hâlde derleme hatası alınacağını söyledi. `gcc -Wall` ile denendi: Bu bir hata değil, "control reaches end of non-void function" uyarısıdır ve program derlenir. Dönen değer büyük olasılıkla tanımsızdır, bu yüzden her yolun bir `return` ile bitmesi yine de gerekir. Faktöriyel örneğinde (Bölüm 13) hocanın kendisinin de gösterdiği gibi, `if` kolu `return` ediyorsa `else` kelimesi şart değildir, önemli olan fonksiyonun sonunda bir `return` bulunmasıdır.

Hoca son olarak `findDistance`, `enUygunBul` ya da `matris_doldur` gibi fonksiyonların ayrı bir C dosyasına yazılıp başlık dosyalarıyla başka programlarda kullanılabileceğini, bunun gelecek dönemin konusu olduğunu söyledi.

11. Doktor ve Teşhis Senaryosu

11.1 Veri Yapısı

Problem şudur: `D` doktor ve `H` hasta vardır. Doktorların hastalara koyduğu teşhisler verilir. Hastalıklar 1 ile 100 arasında kodludur. Doktor 1, hasta 1 için 1 nolu hastalık, doktor 2 aynı hasta için 3 nolu hastalık diyebilir. Teşhisler `D × H`'lik bir matriste saklanır. Her hastanın gerçek teşhisi de `H` elemanlı `dogru` dizisinde tutulur. Hoca bu dizi için ground truth adının da kullanılabileceğini söyledi.

> **Hocanın vurgusu:** "D doktorun H hastaya koyduğu teşhisler saklanacak" denildiğinde aklınıza matris gelmelidir. Veri yapısı neyi gerektiriyorsa ona göre dizi, matris ya da 3 boyutlu matris seçilir.

11.2 En Başarılı Doktor

Soruyu iki parçaya ayırmak gerekir:

1. Her doktorun başarısını (doğru teşhis sayısını) bul ve bir dizide sakla.
2. Bu dizinin en büyük elemanının indisini bul.

Birinci adım için `temp` dizisi tutulur. Boyutu doktor sayısı kadardır. `i`. doktorun `j`. hastaya verdiği teşhis `dogru[j]`'ye eşitse sayaç artar. Sayacı önerirken bir öğrenci geçici bir `t` değişkeni tanımlamayı söyledi. Hoca, `t`'nin her doktor için sıfırlanması gerektiğini ekledi. İkinci adım için önceki derste yazılan en büyüğü bulma fonksiyonu kullanılır. Hoca o fonksiyonu indis döndürecek şekilde değiştirdi. Dönen indis, en iyi doktorun numarasıdır.

Tam program, derste yazılan hâliyle şöyledir:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define DOKTOR 3
#define HASTA 5
#define HASTALIK 10   /* teshis kodlari 1..HASTALIK */

void matris_doldur(int M[][HASTA]);
int enBuyuk(int dizi[], int n, int secim);
int enIyiDoktor(int M[][HASTA], int dogru[]);

int main(void)
{
    int D[DOKTOR][HASTA];   /* D[i][j]: i. doktorun j. hastaya teshisi */
    int dogru[HASTA];       /* her hastanin dogru teshisi */
    int i;

    srand(time(NULL));

    matris_doldur(D);
    printf("\n");
    for (i = 0; i < HASTA; i++) {
```

```
        dogru[i] = 1 + rand() % HASTALIK;
        printf("%3d ", dogru[i]);
    }
    printf("\n");

    i = enIyiDoktor(D, dogru);
    printf("En iyi doktor = %d\n", i);
    return 0;
}

void matris_doldur(int M[][HASTA])
{
    int i, j;
    for (i = 0; i < DOKTOR; i++) {
        for (j = 0; j < HASTA; j++) {
            M[i][j] = 1 + rand() % HASTALIK;
            printf("%3d ", M[i][j]);
        }
        printf("\n");
    }
}

/* secim == 1 ise en buyugun, degilse en kucugun indisini dondurur */
int enBuyuk(int dizi[], int n, int secim)
{
    int i, indis = 0;
    for (i = 1; i < n; i++) {
        if (secim == 1) {
            if (dizi[i] > dizi[indis])
                indis = i;
        } else {
            if (dizi[i] < dizi[indis])
                indis = i;
        }
    }
    return indis;
}

int enIyiDoktor(int M[][HASTA], int dogru[])
{
    int temp[DOKTOR];   /* her doktorun dogru teshis sayisi */
    int i, j, t;

    for (i = 0; i < DOKTOR; i++) {
        t = 0;
        for (j = 0; j < HASTA; j++) {
            if (M[i][j] == dogru[j])
                t++;
        }
        temp[i] = t;
    }
    return enBuyuk(temp, DOKTOR, 1);
}
```

Dizi parametresinde eleman sayısı yazılmadı, çünkü `dogru` dizisinin `HASTA` elemanlı olduğu bilinmektedir. Program çalıştırıldığında hoca sonucu kontrol etti: Bir denemede en iyi doktorun iki teşhisi (9 ve 4 kodlu) tutmuştu, diğer doktorun yalnız biri tutmuştu. Bir başka denemede hiçbir doktorun teşhisi tutmamıştı ve yine ilk doktor seçildi. Karşılaştırma `>` ile yapıldığı için eşitlikte ilk görülen kalır. Tam eşitlik durumu sabit veriyle de denendi ve sonuç 0. doktor oldu.

Soru: En büyüğü bulmak için bir sıralama algoritması kullanmak daha verimli olmaz mı? **Ce-**

vap: Sürekli en büyük ve en küçük aranacaksa diziyi bir kez sıralamak belki daha etkili olabilir. Ama sıralama en az iki döngü gerektirir ve karmaşıklık n^2'ye gider. En büyüğü bulmak ise tek döngüde, en kötü ihtimalle n adımda yapılır.

> **Hocanın vurgusu:** Sıralama gerekli değilse yazmak çok kötü bir çözümdür. "En iyi çözümü bulun" denirse en büyüğü bulma algoritması yazılmalıdır. Sıralama yaparsanız puanınız çok düşer.
>
> **Ders sonrası not:** Hoca sıralamanın en az iki döngü ve n^2 gerektirdiğini söyledi. Bu, derste görülen basit algoritmalar için geçerlidir. Doğrusu büyük olasılıkla şudur: Birleştirmeli sıralama gibi algoritmalar $n \log n$'de çalışır. Ama bu da en büyüğü bulmanın n adımından daha yavaştır, yani hocanın sonucu değişmez.

11.3 Birbirine En Çok Benzeyen İki Doktor

İkinci soru şudur: Birbirine en çok benzer sonuçları veren iki doktoru bulun. Hoca bunun neyi hatırlattığını sordu: kategorik uzaklığı. Teşhis kodları arasındaki farkın anlamı yoktur, eşit olup olmadıkları önemlidir. Tek bir matris olduğu için çöpçatan örneğinden farklı olarak `j`, `i + 1`'den başlar. Algoritma için iç içe iki `for` ve özellikler için üçüncü bir `for` gerekir.

Soru: Her ikilinin benzerliğini tutmak için ek bir dizi gerekir mi? **Cevap:** Gerekmez. Tek bir değişken tutulur, daha yüksek bir benzerlik bulununca güncellenir.

Hoca bu kodun yazılmasını öğrencilere bıraktı. Aynı veri üzerinden iki çalışma sorusu daha verdi:

- **En az konulan teşhis:** En nadir konulan teşhisi bulun.
- **Çoğunluk oyu:** Teşhis, doğru dizisi yerine her hasta için doktorların en çok verdiği teşhis olarak seçilseydi kimlere yanlış teşhis konurdu? Bu, derste daha sonra ele alınacak.

12. Özyinelemeli (Recursive) Fonksiyonlar

12.1 Tanım

Bir fonksiyon kendi içinden kendini çağırıyorsa ya da başka bir fonksiyonu çağırıp o da kendisini çağıran fonksiyonu çağırıyorsa bu fonksiyon özyinelemelidir. İlki doğrudan, ikincisi dolaylı olarak kendini çağırmadır. Bir fonksiyonun başka bir fonksiyonu çağırması tek başına özyineleme değildir.

12.2 Deneme: Stack Gerçekten Doluyor mu?

Hoca dersin başındaki stack anlatımını sınamak için durma koşulu olmayan bir fonksiyon yazdı. Kaç kez çağrıldığını görmek için bir sayaç ekledi. Sayaç sıradan bir yerel değişken olsaydı her çağrıda sıfırlanırdı. Bu yüzden `static` yapıldı. `static` değişken bir kez ilk değer alır ve kaldığı yerden devam eder. Hoca `static`'in gelecek dönem görüleceğini söyledi.

```c
#include <stdio.h>

void f(void);

int main(void)
{
    f();
    return 0;
}

void f(void)
{
    static int x = 0;   /* bir kez ilk deger alir, her cagrida sifirlanmaz */
    x++;
    printf("deneme %d\n", x);
    f();
}
```

Program çalışır, ekrana yazar ve bir süre sonra işletim sistemi "artık yeter" der. Her çağrıda dönüş satırı stack'e eklendiği için stack sürekli dolar ve hiç boşalmaz. Hocanın bilgisayarında sayı **64905**'te durdu. Hoca kendi bilgisayarında deneyenlerin farklı bir sayıya ulaşacağını söyledi.

> **Ders sonrası not:** Hoca bu sayıyı bilgisayarının RAM miktarıyla ilişkilendirdi, 2-4 GB RAM'de sayının çok daha küçük kalacağını söyledi. Büyük olasılıkla asıl belirleyici, RAM miktarı değil, işletim sisteminin programa ayırdığı stack boyutudur (Windows'ta varsayılan büyük olasılıkla 1 MB'tır). Aynı program bu notu hazırlarken Linux'ta 8 MB stack sınırıyla derlenip çalıştırıldı ve 523197 çağrıda sonlandı (segmentation fault). `gcc -Wall` bu fonksiyon için bir de "infinite recursion" uyarısı veriyor.

Hoca ardından çok önemli bir karşılaştırma yaptı. `f`'in içindeki kendini çağıran satır kaldırılıp fonksiyon bir döngüden 100 bin kez çağrılsın:

```c
#include <stdio.h>

void f(void);

int main(void)
{
    int i;
    for (i = 0; i < 100000; i++)
        f();
    return 0;
}

void f(void)
{
    static int x = 0;
    x++;
    printf("deneme %d\n", x);
}
```

Fonksiyon yine 100 bin kez çağrılır, ama hiçbir sorun çıkmaz. Hoca 1 milyon yazılsa da sorun olmayacağını söyledi. Çünkü her çağrı tamamlanır, bilgiler stack'ten çıkar, `for`'a dönülür ve yeni çağrı yapılır. Stack dolar, boşalır, dolar, boşalır, hiçbir şey üst üste binmez. İç içe çağrıda ise hiçbir çağrı bitmediği için stack birikerek dolar. Fark budur.

12.3 Neden Özyineleme?

Soru: Bir fonksiyon neden kendini çağırır? **Cevap:** Bir öğrenci faktöriyeli örnek gösterdi. Hoca bunu şöyle açtı: $5!$ hesaplamak için $5 \times 4 \times 3 \times 2 \times 1$ yapılır. Ama $4!$ problemi daha önce çözüldüyse tekrar $4 \times 3 \times 2 \times 1$ yapmaya gerek yoktur: $5! = 5 \times 4!$ tek çarpmayla bulunur. Genel olarak $n! = n \times (n-1)!$ yazılır. Yani fonksiyon, problemin daha önce çözülmüş parçasını kullanarak yeni çözümü elde eder.

> **Ders sonrası not:** Hoca $5!$'in normalde 5 çarpma gerektirdiğini söyledi. $5 \times 4 \times 3 \times 2 \times 1$ yazımında 4 çarpma vardır, 5 çarpma ise `f = 1` ile başlayıp 1'den 5'e giden döngüde olur. Tek çarpma kazancı da yalnızca $4!$ bir yerde hazır tutuluyorsa geçerlidir: Özyinelemeli `fakt`, $4!$'ü de yeniden hesapladığı için toplam çarpma sayısı azalmaz.

Hoca bu yaklaşımın kriptolojide çok kullanıldığını, örneğin asal sayı bulmada iç içe geçen problemler (overlapping problem) yaklaşımıyla algoritmalar kurulduğunu söyledi. Bilgisayar ne kadar güçlüyse o kadar büyük asal sayılar bulunabilir.

> **Ders sonrası not:** Hoca çok büyük bir asal sayıya sahip olmanın daha küçük asalları bulmayı "çerez gibi" kolaylaştırdığını ve bu sayede başkasının şifresinin daha rahat çözüleceğini söyledi. Doğrusu büyük olasılıkla şudur: Büyük bir asalı bilmek küçük asalları bulmaya doğrudan yardım etmez. RSA gibi şifrelemelerin güvenliği, iki büyük asalın çarpımını çarpanlarına ayırmanın zorluğuna dayanır.

13. Özyinelemeli Faktöriyel

13.1 Bağıntı ve Durma Koşulu

Önce bilinen iteratif hâl yazıldı: `f = 1`, `i` 1'den `n`'e giderken `f *= i`, sonunda `return f`.

Özyinelemeli kod yazmak için önce **özyineleme bağıntısı** (recurrence relation) gerekir. Bağıntıyı yalnızca $f(n) = n \cdot f(n-1)$ diye yazıp açalım:

$$f(5) = 5{\cdot}f(4),\ f(4) = 4{\cdot}f(3),\ f(3) = 3{\cdot}f(2),\ f(2) = 2{\cdot}f(1),\ f(1) = 1{\cdot}f(0),\ f(0) = 0{\cdot}f(-1),\ \ldots$$

Hoca "sizce doğru gidiyor muyum?" diye sordu. Bir yerde iş sakata düşmüştür: $f(0) = 0{\cdot}f(-1)$ sonucu sıfıra götürür ve açılım eksi sayılara doğru sonsuza kadar sürer. Demek ki ikinci aşama da **durma koşulunu** yazmaktır. Bağıntı parçalı bir fonksiyona dönüşür:

$$f(n) = \begin{cases} n \cdot f(n-1) & n \geq 2 \\ 1 & n = 0 \text{ veya } n = 1 \end{cases}$$

Matematikte bildiğimiz $0! = 1! = 1$ bilgisi koda yansıtılmalıdır.

> **Hocanın vurgusu:** En kilit nokta şudur: Bir problemin çözümü kendinden küçük alt problemlerin çözümüne bağlıysa özyinelemeyle çözülebilir. İki aşaması vardır: özyineleme bağıntısını oluşturmak ve durma koşulunu belirlemek. Durma koşulu yazılmazsa hem stack overflow olur hem de yanlış sonuca gidilir.

13.2 Durma Koşulunun Unutulması

Hoca önce kodu koşulsuz yazdı, `return n * fakt(n - 1);` ve her çağrıda stack'e neyin eklendiğini izledi: 5 çarpı `f(4)` beklemede kalır, sonra 4 çarpı `f(3)`, 3 çarpı `f(2)`, 2 çarpı `f(1)`, 1 çarpı `f(0)`, 0 çarpı `f(-1)` diye devam eder. `fakt(3)` gibi küçük görünen bir çağrı bile yine durmadı ve sayaç gene 64905'te kaldı. `n` değeri de yazdırılınca sayı 64902 oldu.

> **Ders sonrası not:** Hoca ekrandaki `n` değerini görünce bir yerde işaretin değişmiş olabileceğini söyledi. `n` her çağrıda 1 azalır: 3, 2, 1, 0, -1, -2, ... Yani negatif değerler taşmadan değil, durma koşulu olmadığı için görülür. Sayaç onbinlerde kalırken `n` için taşma büyük olasılıkla söz konusu olmaz.

13.3 Son Kod

```c
#include <stdio.h>

int faktIteratif(int n);
int fakt(int n);

int main(void)
{
    printf("%d\n", fakt(3));
    printf("%d\n", faktIteratif(3));
    return 0;
}

int faktIteratif(int n)
{
    int i, f = 1;
    for (i = 1; i <= n; i++)
        f *= i;
    return f;
}
```

```c
int fakt(int n)
{
    if (n >= 0 && n < 2)
        return 1;
    return n * fakt(n - 1);
}
```

Çıktı 6'dır. `gcc -Wall -std=c89 -pedantic` ile uyarısız derlendi. `fakt(0)` 1, `fakt(5)` 120 verir.

Koşulun yazımı derste tartışıldı. Hoca eksi değerler için 1 dönmesin diye $n \geq 0$ şartını da koydu.

Soru: `n >= 0` yerine `n > 0` yazsak bir işlem azalmaz mı? **Cevap:** $0!$ de sorulabilir, $0!$ için 1 dönmesi gerekir. Bu yüzden eşittir kalmalıdır.

Soru: `else` yazmadık, yine de çalıştı. Neden? **Cevap:** `if` koşulu doğruysa `return` edilir ve fonksiyon biter. Koşul doğru değilse zaten `else`'in olduğu yere gelinmiş olur. Bu yüzden `else` gerekmez.

> **Ders sonrası not:** Bu kodda `n` negatif verilirse `if` koşulu sağlanmaz ve `n * fakt(n - 1)` sonsuza kadar devam eder. Yani stack overflow tekrar görülür. Negatif girişler fonksiyon çağrılmadan önce engellenmelidir ya da koşul `if (n < 2) return 1;` olarak yazılmalıdır.

14. Üs Alma: Problemi Küçültmenin Kazancı

Hoca bu yaklaşımın kazancını kısa bir örnekle gösterdi: x^n hesabı. İteratif hesapta $n-1$ çarpma yapılır. Ama $x^{n/2}$ biliniyor olsaydı $x^n = x^{n/2} \cdot x^{n/2}$ tek bir çarpmayla bulunurdu. $x^{n/2}$ için $x^{n/4}$, onun için $x^{n/8}$ gerekir ve her biri birer çarpmayla bulunur. Böylece $n-1$ yerine $\log n$ mertebesinde işlem yapılır.

Verilen: x^{1024}. **İteratif:** 1023 çarpma. **Yarıya bölerek:** $x^2, x^4, x^8, \ldots, x^{1024}$ zinciri, $\log_2 1024 = 10$ çarpma.

> **Ders sonrası not:** Hoca iteratif algoritma için önce $n-1$ dedi, sonra x^{1024} için 1024 işlem dedi. Doğru sayı $n-1 = 1023$'tür (python ile sayıldı). Ayrıca 10 çarpma sonucu, $x^{n/2}$'nin bir kez hesaplanıp saklanmasını gerektirir. Kodda `us(x, n/2) * us(x, n/2)` diye iki kez çağrılırsa çarpma sayısı yine 1023 olur (python ile sayıldı). `n` tekse fazladan bir çarpma daha gerekir.

Bu algoritmanın analizine sonraki derste devam edilecek.

Dersin Toparlaması

Bu derste önce fonksiyon çağrısının arka planı görüldü: parametreler kopyalanır, gövde çalışır, değer dönüş yerine düşer. Dönüş adresleri ve yedeklenen bilgiler, işletim sisteminin prosese ayırdığı sınırlı stack bölgesinde ilk giren son çıkar mantığıyla tutulur. Bunun bedeli küçük bir zaman kaybı ve stack overflow riskidir. Sonra en yakın çift problemi tek boyuttan iki boyuta ve k boyuta genelleştirildi. Manhattan, Öklid ve kategorik uzaklıklar görüldü. Aynı mantık çöpçatan ve doktor senaryolarıyla koda döküldü. Bu senaryolarda asıl beceri, verilen metinden uygun veri yapısını seçmekti.

Dersin sonunda özyinelemeye girildi. Stack'in gerçekten dolduğu deneyle gösterildi, faktöriyel üzerinden özyineleme bağıntısı ve durma koşulu görüldü, üs alma örneğiyle de problemi küçültmenin kazancı işaret edildi. Bir sonraki derste özyinelemeli algoritmaların analizine devam edilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.